

Lista de Exercícios 01 – Lógica com GDScript

29/09/2026 • Tarik Ponciano

Nome: _____ Turma: _____ Data: _____

Instruções gerais

Esta lista pratica a Aula 01 em 21 questões e 3 desafios. Cada questão vira uma cena + script em `lista01/`, recebe os dados pelo Inspetor (`@export`) e mostra o resultado no painel Saída.

Regras

1. **Uma cena + um script por questão**, com o mesmo nome, na pasta `lista01/`. Exemplo: `q07_media.tscn` + `q07_media.gd`.
2. **Nó raiz** `Node` em todas as questões.
3. **Terminal da turma instalado** no projeto (Autoload com o nome `Terminal`), como no slide da Seção 01.
4. **Entradas:** via `@export` nos Níveis 1 a 3, com os valores padrão da questão; pelo Terminal no Nível 4. Na correção, os valores serão trocados: o programa precisa funcionar com qualquer valor válido.
5. **Saída:** `print()` ou `Terminal.escrever()` – os dois funcionam do mesmo jeito.
6. **Tipagem obrigatória:** toda variável com `: Tipo` ou `:=`. Única exceção: Q14.
7. **Sem `if`, `for` ou `while`**. Única exceção: Q19, que usa o `if` do modelo do slide. Todo o resto sai de operadores e funções da aula.
8. **Nomes em snake_case**, sem acentos nem espaços; constantes em CAIXA_ALTA.

Passo a passo de cada questão

1. **Cena > Nova Cena > Outro Nó > Node**, e salve como `lista01/qNN_nome.tscn`.
2. Anexe um script com o mesmo nome e comece pelo modelo abaixo.
3. Declare as entradas com `@export` e os valores padrão da questão (no Nível 4, pergunte pelo Terminal).
4. Rode com **Cmd+R** e compare a saída com o exemplo, caractere por caractere.
5. Troque os valores no Inspetor e rode de novo para testar outros casos.
6. Responda às perguntas como comentários no fim do script.

Modelo de script

CÓDIGO GDSCRIPT

```
extends Node
# Q07 · Média do aluno
# Aluno(a): [seu nome]

# Entradas: edite no Inspetor
@export var nota1: float = 6.0
@export var nota2: float = 7.5

# Constantes, se a questão pedir
# const NOME_DO_JOGO := "Dungeon Lógica"

func _ready() -> void:
    # 1. Calcular
    # 2. Mostrar na Saída
    pass

# Respostas às perguntas:
# ...
```

Como ler cada questão

MARCAÇÃO	O QUE FAZER
Entradas	Declare essas variáveis com <code>@export</code> , com esses valores padrão
Saída esperada	É o que deve aparecer na Saída com os valores padrão
Pergunta	Responda como comentário no fim do script
Teste também	Rode de novo com esses valores, trocando-os no Inspetor
Extra	Opcional, para quem terminar antes

Níveis e onde revisar

NÍVEL	QUESTÕES	FOCO	REVISE NOS SLIDES
1	Q01-Q05	Saída, variáveis, constantes, annotations e ciclo de vida	Seções 02, 03 e 04
2	Q06-Q13	Operadores aritméticos e lógicos, funções matemáticas	Seção 06
3	Q14-Q18	Tipos, conversões, arredondamento e formatação	Seções 05 e 07
4	Q19-Q21	Entrada pelo Terminal da turma e pelo teclado	Seção 08
Desafios	D1-D3	Problemas de jogo que juntam tudo	Todas

Travou?

- **Erro vermelho?** Leia a mensagem no painel Depurador (Debugger) > Erros e a linha marcada no editor.
- **Resultado estranho?** Coloque um `print()` no meio do cálculo para ver o valor de cada variável.
- **Confira as armadilhas da aula:** `print()` junta sem espaço, `7 / 2` dá 3, texto + número é erro e `to_int()` de texto inválido dá 0.

Antes de entregar

- ☐ Cada questão tem `.tscn` e `.gd` com o mesmo nome em `lista01/`
- ☐ Todas as cenas rodam com Cmd+R sem erros
- ☐ A Saída bate com o exemplo, usando os valores padrão
- ☐ Testei cada questão com pelo menos um outro valor no Inspetor
- ☐ Todas as variáveis têm tipo e nome em snake_case
- ☐ As perguntas estão respondidas como comentários

Entrega: [forma de entrega – ex.: commit no repositório da turma] até [data].

Nível 1 – Saída e variáveis

O Nível 1 treina mostrar informações, declarar variáveis com tipo e organizar o Inspetor.

Q01 • Boas-vindas

Mostre uma tela de boas-vindas na Saída.

- **Entrada:** `@export var nome_jogador: String = "Ana"`
- **Constante:** `const NOME_DO_JOGO := "Dungeon Lógica"`
- Use `print()`, `prints()` em pelo menos uma linha e `print_rich()` para destacar o nome do jogo (negrito ou cor).

SAÍDA ESPERADA

```
=====
Bem-vindo(a) a Dungeon Lógica!
Jogador: Ana
Boa sorte na sua jornada
=====
```

Q02 • Cartão do estudante

Declare uma variável exportada de cada tipo primitivo e mostre um cartão.

- **Entradas:** `nome: String = "Ana"`, `idade: int = 16`, `altura: float = 1.65`, `estudante: bool = true`

SAÍDA ESPERADA

```
--- CARTÃO ---
Nome: Ana
Idade: 16 anos
Altura: 1.65 m
Estudante? true
```

Pergunta: mude a altura para 2 no Inspetor e rode. O que aparece? Por quê?

Q03 • Tempo de sessão

Meça quanto tempo a cena ficou aberta e quantos quadros foram desenhados.

1. Em `_ready()`, mostre `Sessão iniciada`.
2. Em `_process(delta)`, some `delta` em `tempo_total` e 1 em `quadros`. **Sem `print()` aqui.**
3. Ainda no `_ready()`, espere o usuário com `await Terminal.pausar("Tecle Enter para encerrar")`. Enquanto isso, o `_process` continua rodando.
4. Depois da pausa, mostre o resumo com a média de quadros por segundo (`quadros / tempo_total`).

SAÍDA ESPERADA

```
Sessão iniciada
Sessão encerrada: 5.2 s, 312 quadros, média de 60.0 quadros/s
```

Os números variam a cada execução.

Q04 • Item do inventário

Crie a ficha de um item com as annotations da aula, em dois grupos: **Básico** (nome, raridade, descricao) e **Atributos** (preco, peso, empilhavel).

VARIÁVEL	TIPO	ANNOTATION	PADRÃO
nome	String	@export	"Espada Curta"
raridade	String	@export_enum("Comum", "Raro", "Épico", "Lendário")	"Raro"
descricao	String	@export_multiline	"Leve e afiada."
preco	int	@export_range(0, 9999)	150
peso	float	@export_range(0.0, 50.0, 0.5)	3.5
empilhavel	bool	@export	false

SAÍDA ESPERADA

```
[Raro] Espada Curta
Leve e afiada.
Preço: 150 moedas | Peso: 3.5 kg | Empilhável: false
```

Pergunta: tente digitar 20000 no preço pelo Inspetor. O que acontece?

Q05 • Constantes e escopo

Descubra as regras de `const` e de escopo provocando erros de propósito.

1. Crie `const VIDA_MAXIMA := 100` e `const GRAVIDADE := 9.8` e mostre as duas no `_ready()`.
2. No `_ready()`, tente `VIDA_MAXIMA = 200`. Copie a mensagem de erro do editor como comentário no script.
3. Crie `var bonus := 50` dentro do `_ready()` e tente mostrá-la no `_exit_tree()`. Anote o erro.
4. Conserte o item 3 transformando `bonus` em variável membro. Explique em um comentário por que agora funciona.

Entregue o script consertado, com os erros anotados como comentários.

Nível 2 – Operações

O Nível 2 treina operadores aritméticos, lógicos e funções matemáticas, com atenção à divisão inteira.

Q06 • Calculadora completa

- **Entradas:** `a: int = 17`, `b: int = 5`

SAÍDA ESPERADA

```
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3 (divisão inteira)
17 / 5 = 3.4 (divisão real)
17 % 5 = 2
17 ** 5 = 1419857
```

Pergunta: o que a Godot mostra quando `b = 0` ? Anote e explique.

Q07 • Média do aluno

Calcule a média simples e a média ponderada (pesos 1, 2, 3 e 4), com 1 casa decimal.

- **Entradas:** `nota1` a `nota4`, `float` com `@export_range(0.0, 10.0, 0.5)`, valores 6.0, 7.5, 8.0 e 9.0

SAÍDA ESPERADA

```
Média simples: 7.6
Média ponderada: 8.1
```

Pergunta: troque as notas para `int` (6, 7, 8, 9). A média simples continua certa? Por quê?

Q08 • Cronômetro de speedrun

Converta segundos em horas, minutos e segundos usando só `/` e `%`.

- **Entrada:** `segundos_totais: int = 3725`
- **Dica:** `"%02d"` completa com zero à esquerda.

SAÍDA ESPERADA

```
3725 segundos = 1 h 2 min 5 s
Tempo do speedrun: 01:02:05
```

Q09 • Par ou ímpar, sem if

Responda com `true` ou `false` usando apenas operadores.

- **Entrada:** `numero: int = 7`

SAÍDA ESPERADA

```
Número: 7
Resto da divisão por 2: 1
É par? false
É múltiplo de 3? false
```

Pergunta: teste com -7. O resto muda? Use `posmod()` para o resto ser sempre 0 ou 1.

Q10 • Barra de vida

- **Entradas:** `vida_max: int = 120`, `vida: int = 80`, `dano: int = 50`, `cura: int = 15`
 1. Aplique o dano e depois a cura. A vida nunca sai do intervalo de 0 a `vida_max` (use `clampi`).
 2. Calcule a porcentagem de vida com 1 casa decimal.
 3. Monte uma barra de 10 blocos: `preenchidos = roundi(porcentagem / 10)`. Use `"#".repeat(...)` e `"-".repeat(...)`.

SAÍDA ESPERADA

```
Vida: 45/120 (37.5%)  
[####-----]
```

Teste também: dano 500 com cura 0; dano 0 com cura 1000.

Q11 • Portão da masmorra

Mostre cada condição separada e depois o resultado final.

- **Entradas:** `nível: int = 12`, `tem_chave: bool = true`, `energia: int = 15`, `amaldicoado: bool = false`, `tem_reliquia: bool = false`
- **Pode entrar:** nível ≥ 10 e tem a chave e energia > 20 e não está amaldiçoado.
- **Ganha bônus:** nível ≥ 20 ou tem a relíquia.

SAÍDA ESPERADA

```
Nível suficiente? true  
Tem a chave? true  
Energia suficiente? false  
Livre de maldição? true  
PODE ENTRAR? false  
Ganha bônus? false
```

Q12 • Rolagem de dados

Role 2 dados de 6 faces com `randi_range`, some o modificador e compare com a dificuldade. Sorteie também um crítico com 10% de chance: `randf() < 0.1`.

- **Entradas:** `modificador: int = 2`, `dificuldade: int = 10`

SAÍDA ESPERADA

```
Dados: 4 + 5 | Modificador: 2 | Total: 11  
Passou (>= 10)? true  
Crítico? false
```

Rode várias vezes: os dados mudam a cada execução.

Q13 • Distância até o inimigo

Calcule a distância entre o jogador (x_1, y_1) e o inimigo (x_2, y_2) e diga se ele está no alcance.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **Entradas:** `x1`, `y1`, `x2`, `y2`, `alcance`, todos `float`: 0.0, 0.0, 3.0, 4.0 e 6.0

SAÍDA ESPERADA

```
De (0.0, 0.0) até (3.0, 4.0): distância = 5.00  
No alcance (6.0)? true
```

Nível 3 – Tipos e conversões

O Nível 3 treina descobrir tipos, converter sem perder informação e formatar a saída.

Q14 • Detetive de tipos

Uma única variável dinâmica, `var coisa`, recebe valores diferentes. Depois de cada atribuição, mostre o valor e o tipo com `type_string(typeof(coisa))`. Esta é a única questão em que a variável fica sem tipo, de propósito.

- **Valores, nesta ordem:** `10`, `2.5`, `"dez"`, `true`, `null`, `10 / 4`, `10 / 4.0`, `"7" + "3"`
- Antes de rodar, escreva no caderno o tipo que você espera para cada um.

SAÍDA ESPERADA

```
10 → int
2.5 → float
dez → String
true → bool
<null> → Nil
2 → int
2.5 → float
73 → String
```

Q15 • Caixa da loja

Os dados chegam como texto, igual às respostas de `Terminal.perguntar()`. Valide, converta e calcule.

- **Entradas:** `preco_texto: String = "49.90"`, `quantidade_texto: String = "3"`, `desconto_texto: String = "10"` (porcentagem)
1. Valide cada texto com `is_valid_float()` ou `is_valid_int()` e mostre o resultado.
 2. Converta e calcule subtotal, desconto e total, com 2 casas decimais.

SAÍDA ESPERADA

```
Preço válido? true | Quantidade válida? true | Desconto válido? true
Subtotal: R$ 149.70
Desconto (10%): R$ 14.97
Total: R$ 134.73
```

Pergunta: e se `preco_texto` for `"49,90"`, com vírgula? O que acontece com a validação e com o total?

Q16 • Tabela de arredondamento

Mostre o resultado de `int()`, `round()`, `roundi()`, `floor()`, `floori()`, `ceil()` e `ceili()` para o valor exportado.

- **Entrada:** `valor: float = 2.5`
- Antes de rodar, preencha a tabela abaixo. Depois confira, trocando o valor no Inspetor.

FUNÇÃO	2.5	-2.5	7.49	-7.5
<code>int()</code>				
<code>round()</code>				
<code>roundi()</code>				
<code>floor()</code>				
<code>ceil()</code>				

Pergunta: quais funções devolvem `float` mesmo quando o resultado é um número inteiro?

Q17 • Placar formatado

Monte a linha de placar em um único `print`, usando `%` com uma lista `[...]`.

- **Entradas:** `jogador: String = "lia"`, `pontos: int = 4250`, `tempo: float = 83.456`, `precisao: float = 0.875`
- **Use:** `to_upper()`, `"%06d"`, `"%.2f"` e `"%.1f%%"`.

SAÍDA ESPERADA

Jogador: LIA | Pontos: 004250 | Tempo: 83.46 s | Precisão: 87.5%

Q18 • Conserte o código

O script abaixo tem 5 problemas: 2 erros que impedem de rodar, 1 aviso amarelo, 1 variável sem tipo e 1 resultado errado. Conserte todos e explique cada um em um comentário.

CÓDIGO GDSCRIPT

```
extends Node

@export var nome: String = "Lia"
@export var idade: int = 16

func _ready() -> void:
    var dias: int = idade * 365.25
    print("Olá, " + nome + "! Você tem " + idade + " anos.")
    print("Isso dá cerca de " + dias + " dias.")
    var dias_por_ano = dias / idade
    print("Média de dias por ano: " + str(dias_por_ano))
```

Saída depois do conserto:

SAÍDA ESPERADA

Olá, Lia! Você tem 16 anos.
Isso dá cerca de 5844 dias.
Média de dias por ano: 365.25

Nível 4 – Entrada de dados

O Nível 4 recebe dados enquanto o programa roda: pelo teclado e pelo Terminal da turma.

Q19 • Placar no teclado

Raiz `Node`. Aqui o `if` pode ser usado exatamente como no modelo do slide: um `if` por tecla.

AÇÃO	TECLA	EFEITO NOS PONTOS
<code>ui_accept</code>	Enter ou Espaço	+1
<code>ui_up</code>	Seta para cima	+10
<code>ui_down</code>	Seta para baixo	-10, nunca abaixo de 0 (use <code>maxi</code>)
<code>ui_cancel</code>	Esc	zera

Depois de cada tecla, mostre o placar. Sequência Enter, cima, cima, baixo, Esc:

SAÍDA ESPERADA

```
Pontos: 1
Pontos: 11
Pontos: 21
Pontos: 11
Pontos: 0
```

Extra: a linha `pontos += int(Input.is_action_just_pressed("ui_accept"))` soma pontos sem `if`. Explique por que ela funciona.

Q20 • Saudação personalizada

Raiz `Node`. Pergunte o nome com `await Terminal.perguntar()`. Mostre o nome em maiúsculas, quantos caracteres ele tem e um nome de guerreiro com a constante `TITULO := "Destemido"`.

SAÍDA ESPERADA

```
Qual é o seu nome? Ana
Olá, ANA! Seu nome tem 3 caracteres.
Nome de guerreiro: Ana, o Destemido
```

Pergunta: digite “Ana Clara”. A contagem bate com o número de letras? Por quê?

Q21 • Barra de experiência

Raiz `Node`. O programa pergunta 3 vezes quanto XP o herói ganhou, com `perguntar_int`, e mostra o progresso depois de cada resposta.

- `xp_total` é uma variável membro que acumula o XP.
- `nivel = xp_total / 100 + 1` e `faltam = 100 - xp_total % 100`.

Respostas digitadas: `120`, `abc`, `130`, `45`.

SAÍDA ESPERADA

```
XP ganho: 120
XP total: 120 | Nível: 2 | Faltam 80 XP
XP ganho: abc
Digite um número inteiro. Exemplo: 42
XP ganho: 130
XP total: 250 | Nível: 3 | Faltam 50 XP
XP ganho: 45
XP total: 295 | Nível: 3 | Faltam 5 XP
```

Pergunta: o que o Terminal fez quando você digitou `abc`? Agora digite `-500`: o que acontece? Como impedir isso com uma função da aula, sem `if`?

Desafios

Os desafios juntam todos os níveis em problemas de jogo, sem `if`: a lógica sai de `/`, `%`, funções matemáticas e conversões.

D1 • Carteira de moedas

No jogo, 1 ouro = 100 pratas e 1 prata = 100 cobres. Todo valor fica guardado em cobres (`int`).

- **Entradas:** `carteira_cobre: int = 123456`, `preco_espada_cobre: int = 25075`

1. Mostre a carteira e o preço da espada em ouro, prata e cobre.
2. Compre a espada e mostre a nova carteira.
3. Com a carteira original, quantas espadas daria para comprar e quanto sobriaria?

SAÍDA ESPERADA

Carteira: 12 ouro, 34 prata, 56 cobre
Espada: 2 ouro, 50 prata, 75 cobre
Depois da compra: 9 ouro, 83 prata, 81 cobre
Daria para comprar 4 espadas, sobrando 2 ouro, 31 prata, 56 cobre

D2 • Relógio do mundo do jogo

Cada minuto real vale 1 hora no jogo, e o Dia 1 começa às 06:00. Calcule o dia e o horário do jogo e diga se é noite (das 18:00 às 05:59).

- **Entrada:** `minutos_reais: float = 50.5`
- **Dica:** converta tudo para minutos de jogo (`int`) e trabalhe com `/` e `%`.

SAÍDA ESPERADA

50.5 minutos reais = Dia 3 – 08:30
É noite? false

Teste também: 0 (Dia 1 – 06:00), 18 (Dia 2 – 00:00, noite) e 13.25 (Dia 1 – 19:15, noite).

D3 • Atributo de RPG (4d6)

Role 4 dados de 6 faces, descarte o menor e some os 3 restantes. Depois calcule o modificador do atributo com `floori((valor - 10) / 2.0)`.

- **Dica:** o menor dos quatro é `mini(mini(d1, d2), mini(d3, d4))`.

SAÍDA ESPERADA

Dados: 5, 2, 6, 3 (descartado: 2)
Força: 14 | Modificador: 2

Pergunta: calcule o modificador para 8, 9 e 15. Por que usar `floori()` e não `int()`? Pense no valor 9.